

مرجع نظام تاجر

توثيق معماري وعلمي للنظام: تسجيل الحساب، المتجر، الخطط، الصلاحيات، قاعدة البيانات المحلية، Firebase، والمزامنة عبر `sync_queue`.

نسخة مرجعية أولية - Offline First + Firebase

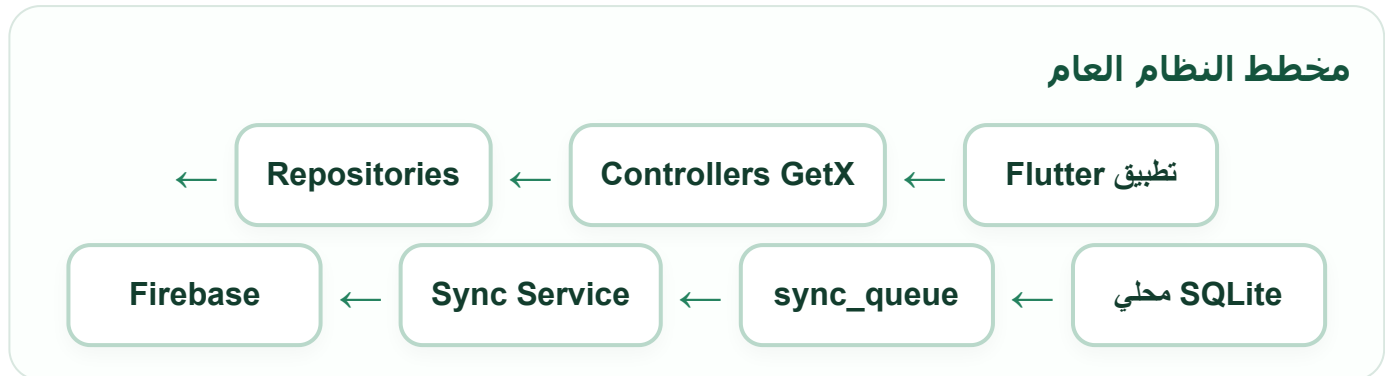
الفهرس

- النظرة العامة على النظام
- هيكلية المشروع البرمجية
- تسجيل الدخول وإنشاء المتجر
- الصلاحيات والأدوار
- الخطط والاشتراكات
- الأنظمة الفرعية: المنتجات، الموردون، المشتريات، العملاء، الفواتير، الديون
- المزامنة مع **Firestore** وشرح **sync_queue**
- مخطط **Schema / ERD**

هذا الملف مبني على الفكرة التي تم الاتفاق عليها: أول تسجيل وتشغيل أساسي يحتاج إنترنت، ثم يعمل النظام محليًا، وبعدها ترفع العمليات إلى **Firestore** عند توفر الاتصال.

1. النظرة العامة على النظام

النظام عبارة عن تطبيق Flutter لإدارة متجر تجاري. يعمل محليًا باستخدام SQLite، ويستخدم Firebase للحسابات، المتاجر، الخطط، الاشتراكات، والنسخة السحابية المركزية.



كيف يعمل هذا المخطط؟

المستخدم يتعامل مع الواجهة. الكنترولر ينفذ منطق الشاشة. المستودع يحفظ البيانات في SQLite. أي عملية تحتاج مزامنة تسجل في sync_queue، ثم SyncService يرفعها إلى Firebase.

Firebase

المصدر المركزي للحسابات، المتاجر، الخطط، الاشتراكات، والنسخة السحابية التي تجمع بيانات المتجر.

SQLite المحلي

المصدر السريع للعمل اليومي. البيع والشراء والإضافة تتم محليًا فورًا حتى بدون إنترنت.

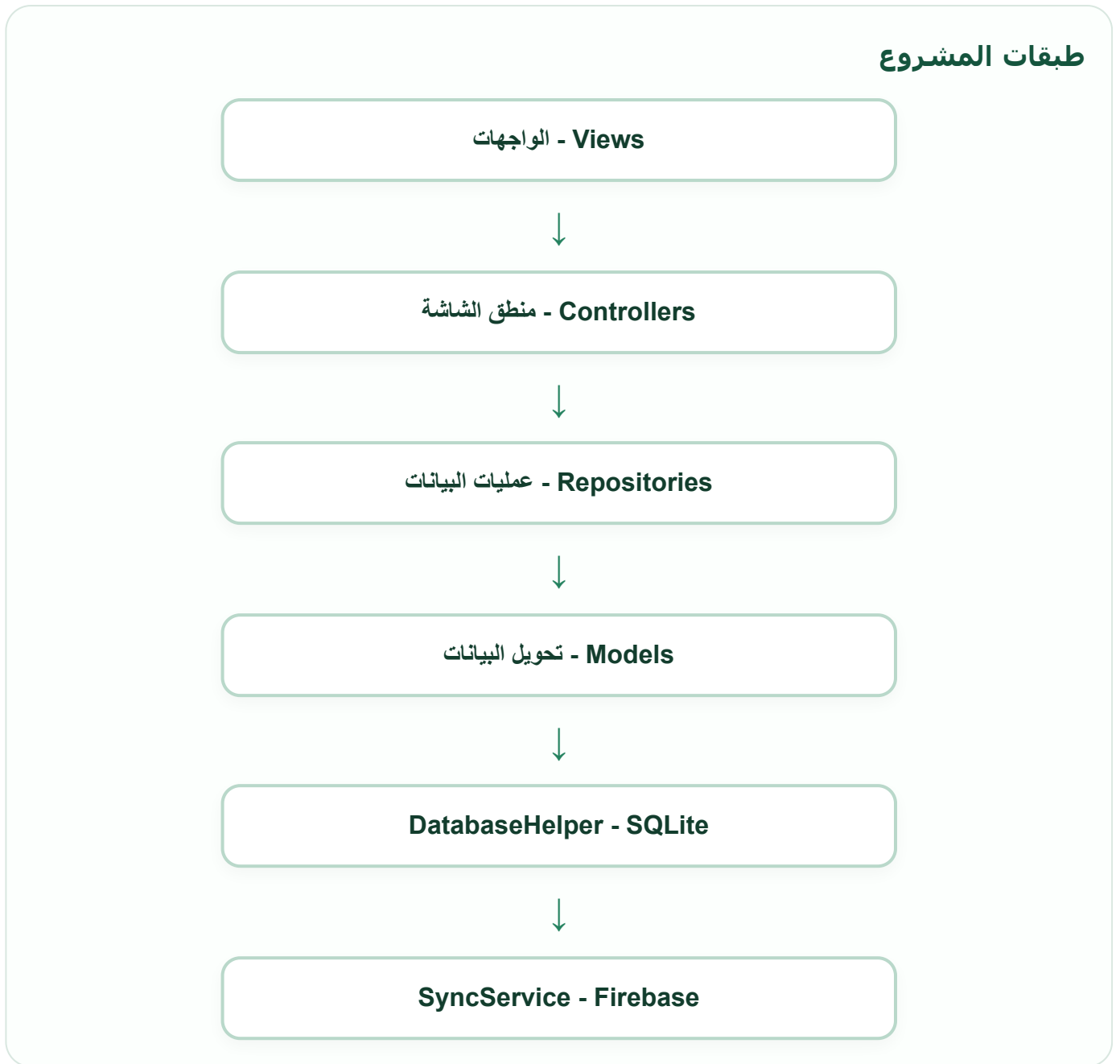
SyncService

المسؤول عن قراءة الطابور، رفع العمليات، إعادة المحاولة، وتحديث الحالة.

sync_queue

طابور العمليات التي لم ترفع بعد. يمنع فقدان البيانات عند انقطاع الإنترنت.

2. هيكلية المشروع البرمجية

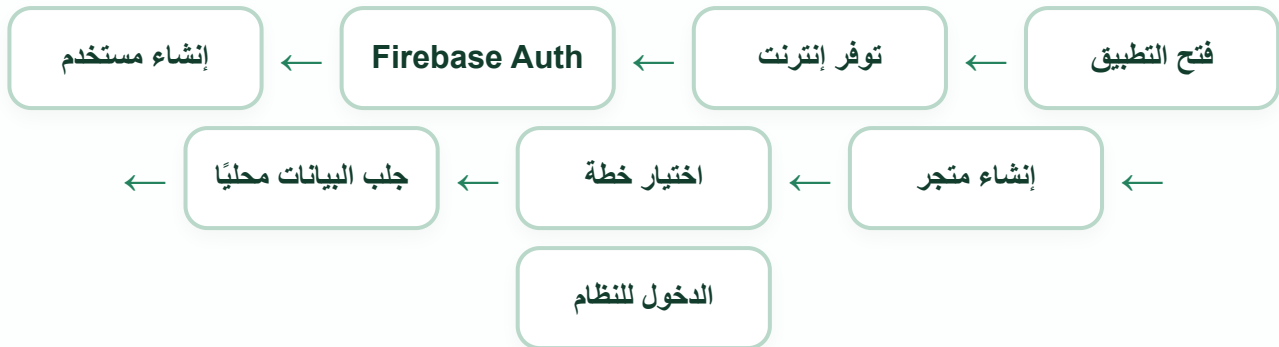


الهدف من هذه الهيكلية أن كل طبقة لها مسؤولية واضحة. الواجهة لا تتعامل مباشرة مع قاعدة البيانات، والـ Repository لا يهتم بشكل الشاشة، والمزامنة لا تختلط بعمليات العرض.

الطبقة	المسؤولية	مثال
Views	عرض الشاشة واستقبال تفاعل المستخدم	Supplierscreen, ProductScreen
Controllers	تجهيز البيانات والتحقق وتنفيذ الأوامر	SupplierController, ProductController
Repositories	قراءة وكتابة SQLite	SupplierRepository, ProductRepository
Models	تحويل Map إلى كائن والعكس	Suppliermodel, ProductModel
DatabaseHelper	إنشاء الجداول والترقيات	version, onCreate, onUpgrade
SyncService	رفع وسحب البيانات من Firebase	قراءة sync_queue

3. تسجيل الدخول وإنشاء المتجر

دورة أول استخدام

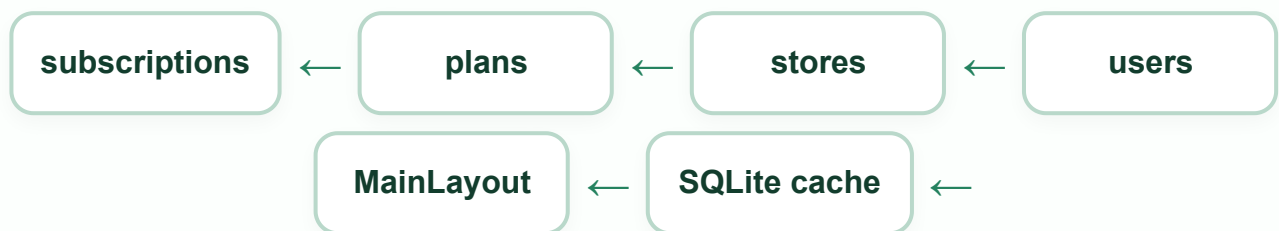


شرح المخطط

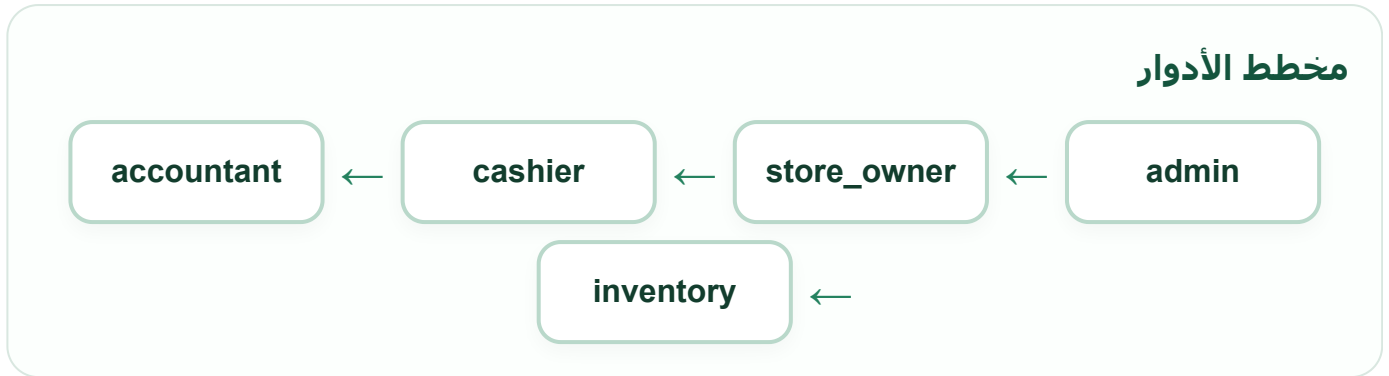
أول مرة يجب أن يكون هناك إنترنت لأن بيانات الهوية والمتجر والخطة يجب أن تبدأ من السحابة. بعد ذلك يتم تنزيل نسخة محلية إلى SQLite حتى يعمل النظام بدون إنترنت.

كلمة المرور لا تحفظ محليًا. الذي يمكن حفظه محليًا هو معرف المستخدم أو token آمن، أما المصادقة الأصلية فهي عبر Firebase Auth.

تسلسل البيانات بعد الدخول



4. الصلاحيات والأدوار



الدور	الصلاحيات المقترحة
admin	إدارة النظام، الخطط، الاشتراكات، مراقبة المتاجر
store_owner	إدارة متجره، الموظفين، المنتجات، التقارير، الصلاحيات
cashier	إنشاء فواتير البيع، البحث عن المنتجات، استلام الدفع
accountant	متابعة الديون، المدفوعات، الفواتير، التقارير المالية
inventory	إدارة المنتجات، التصنيفات، الموردين، المشتريات والمخزون

تطبيق الصلاحيات يجب أن يكون على مستويين: داخل التطبيق لإخفاء الشاشات، وفي Firebase Rules أو Cloud Functions لمنع أي تعديل غير مصرح به.

5. الخطط والاشتراكات



الخطّة تحدد حدود المتجر مثل عدد المنتجات، مدة الاشتراك، والسعر. الاشتراك يربط المتجر بالخطّة ويحدد تاريخ البداية والنهاية والحالة.

العنصر	وظيفته
plans	تعريف الخطّة: الاسم، السعر، حد المنتجات، مدة الخطّة
subscriptions	تسجيل اشتراك المتجر، حالته، تاريخ البداية والنهاية
stores.status	حالة المتجر العامة: نشط، معلق، غير نشط
is_active_status	حالة التنفيع المحلي أو السحابي للمتجر

اقتراح مهم لاحقاً: إضافة plan_id داخل subscriptions حتى يكون الاشتراك مرتبطاً بخطّة محددة بشكل صريح.

6. الأنظمة الفرعية داخل النظام

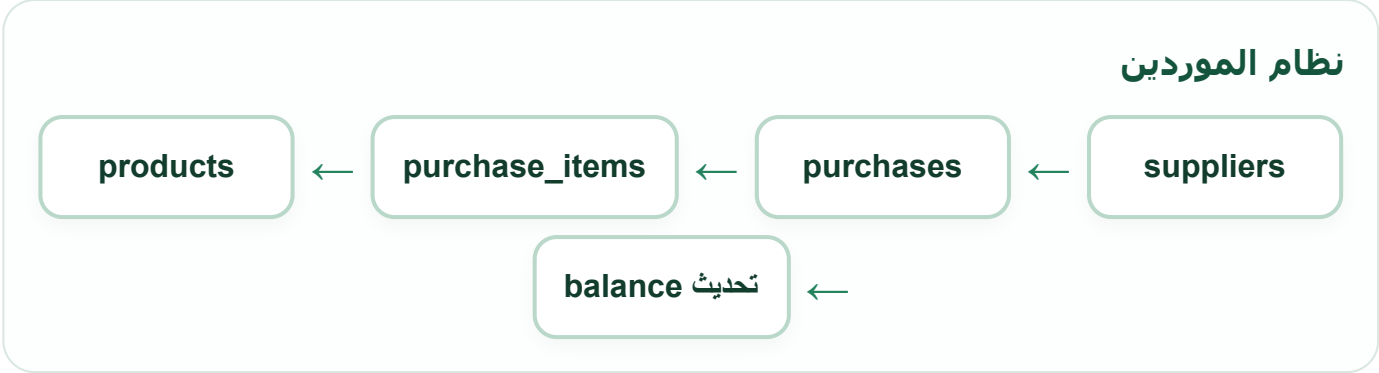
الموردون والمشتريات إدارة المورد، فواتير الشراء، تحديث الرصيد والمخزون.	المنتجات والتصنيفات إدارة الأصناف والباركود والسعر والكمية.	المستخدمون والمتاجر تسجيل الحساب، ربطه بمتجر، تحديد الدور والصلاحية.
المزامنة حفظ محلي أولاً ثم رفع عبر sync_queue عند توفر الإنترنت.	الديون والمدفوعات تحويل المتبقي إلى دين وتوزيع الدفعات على الديون.	العملاء والفواتير إنشاء فواتير البيع وتسجيل تفاصيل الأصناف.

7. المنتجات والمخزون



المنتج مرتبط بمتجر وتصنيف. عند البيع ينقص المخزون من خلال تفاصيل الفاتورة، وعند الشراء تزيد الكمية من خلال تفاصيل الشراء.

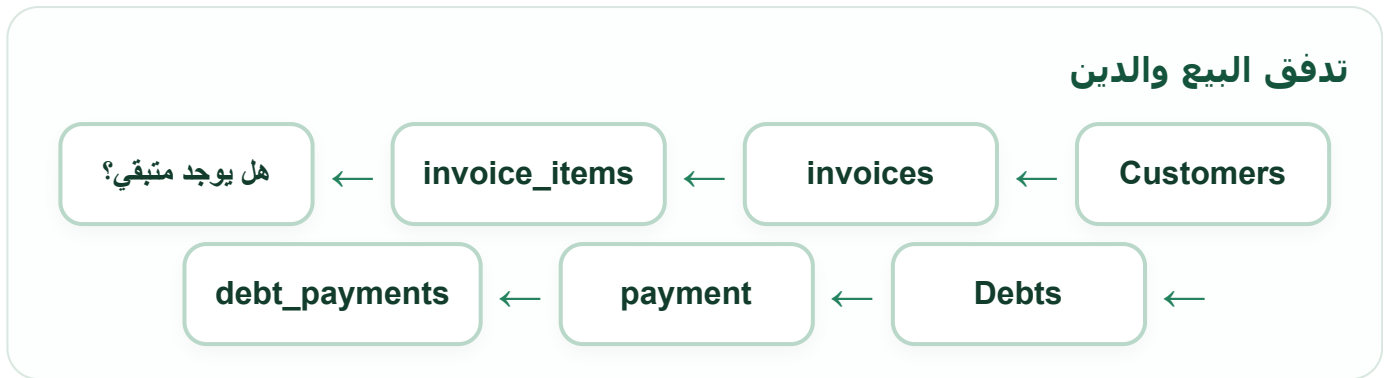
8. الموردون والمشتريات



المورد يحفظ بياناته ورصيده. عند إنشاء شراء أجل أو جزئي يتم تحديث balance للمورد. شاشة الموردين تعرض إجمالي المستحقات من مجموع أرصدة الموردين.

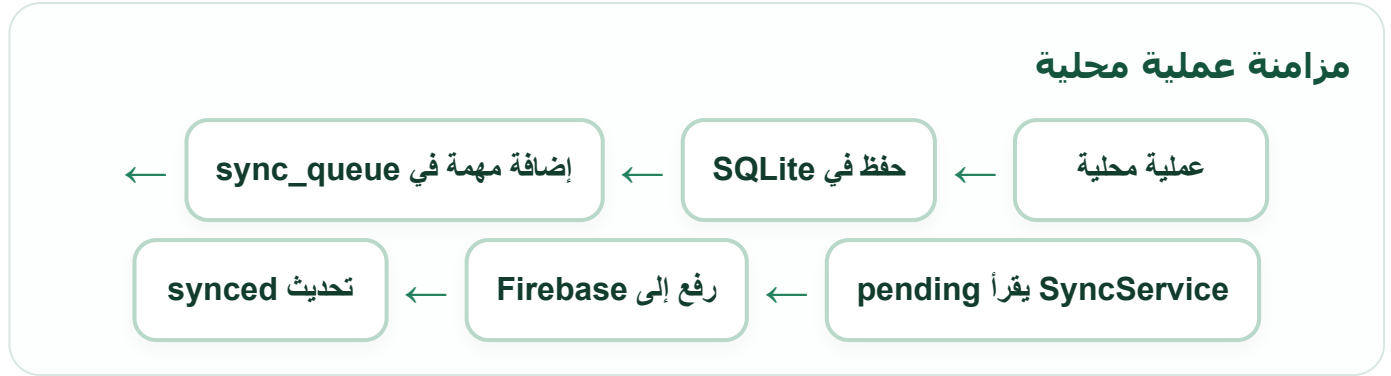
الحقل	المعنى
supplier_name	اسم المورد
phone	رقم الهاتف
address	عنوان المورد
notes	ملاحظات إضافية من الشاشة الجديدة
balance	المبلغ المستحق للمورد أو الرصيد المرتبط به

9. العملاء والفواتير والديون



عند إنشاء فاتورة، تحفظ الفاتورة وتفاصيلها. إذا كان هناك مبلغ متبقٍ يتم إنشاء دين. لاحقًا عند الدفع، يتم تسجيل payment وتوزيع المبلغ على الدين أو عدة ديون عبر debt_payments.

10. المزامنة بالتفصيل

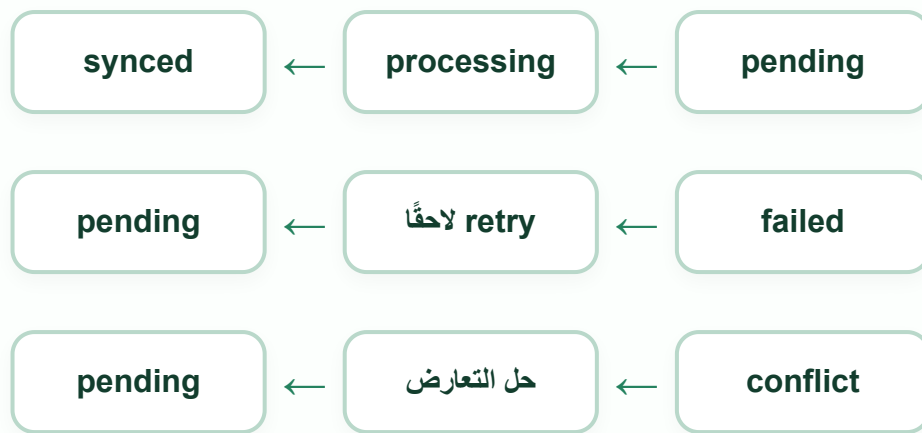


كيف يعمل sync_queue؟

كل عملية إنشاء أو تعديل أو حذف تُسجل كمهمة. المهمة تحتوي اسم الجدول، رقم السجل، نوع العملية، البيانات payload، الحالة، عدد المحاولات، ومفتاح idempotency لمنع التكرار.

الحقل	وظيفته
table_name	اسم الجدول الذي حدثت فيه العملية
record_id	معرف السجل المحلي
operation_type	create أو update أو delete
payload	نسخة JSON من البيانات المراد رفعها
status	pending, processing, synced, failed, conflict
priority	ترتيب أهمية الرفع
attempts	عدد محاولات الرفع
next_retry_at	وقت إعادة المحاولة بعد الفشل
idempotency_key	منع تكرار نفس العملية في Firebase

حالات المزامنة



11. ترتيب رفع البيانات

الترتيب مهم جدًا لأن هناك جداول تعتمد على جداول أخرى. لا يجوز رفع تفاصيل الفاتورة قبل رفع الفاتورة نفسها.

ترتيب آمن للمزامنة

users / stores / subscriptions



categories



products



suppliers / Customers



purchases / invoices



purchase_items / invoice_items



Debts



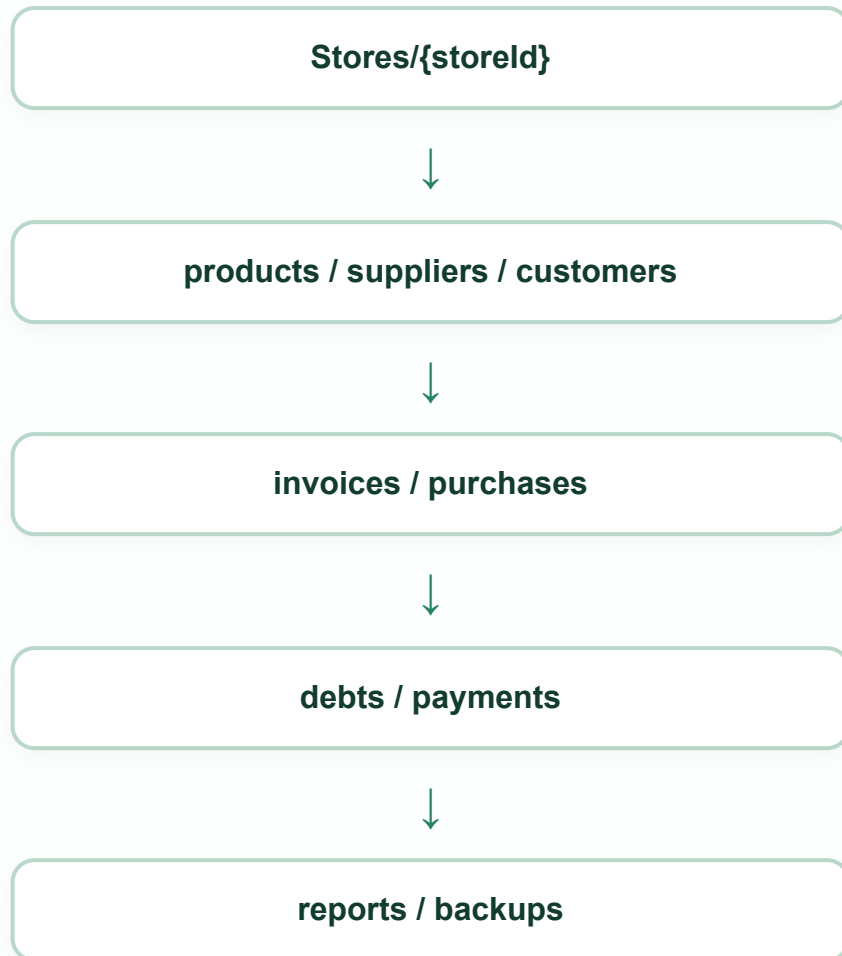
payment



debt_payments

12. Firestore المقترح

بنية Firebase

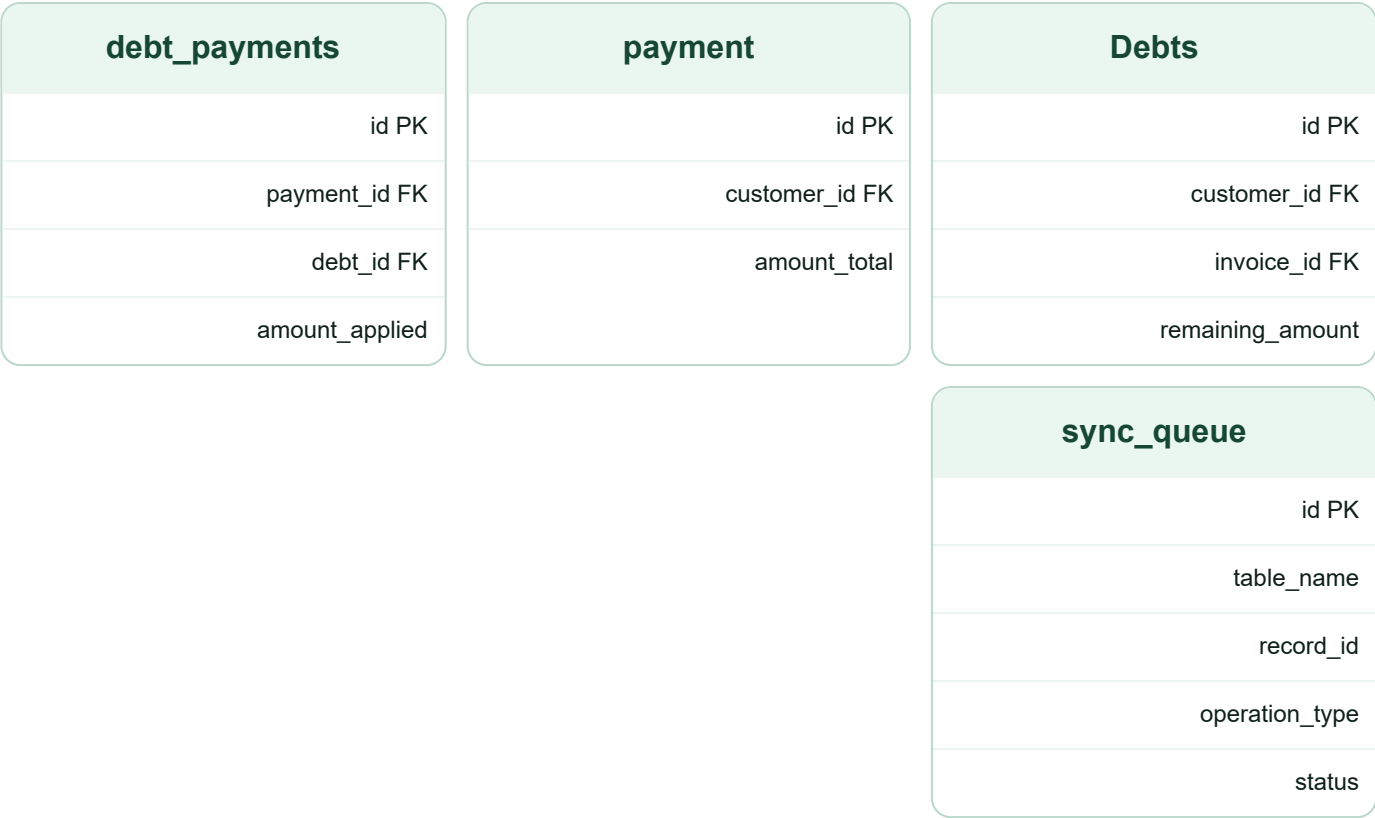


الأفضل وضع بيانات كل متجر تحت مستند المتجر. هذا يجعل قواعد الأمان أسهل: المستخدم لا يستطيع قراءة أو تعديل إلا بيانات المتجر المرتبط به.

```
Stores/{storeId}/products/{productId}
Stores/{storeId}/suppliers/{supplierId}
Stores/{storeId}/invoices/{invoiceId}
Stores/{storeId}/purchases/{purchaseId}
Stores/{storeId}/debts/{debtId}
```

مبسٹ Schema / ERD .13

plans	stores	users
id PK	id PK	uid PK
name	owner_id FK	full_name
price	store_name	email
product_limit	status	role
products	categories	subscriptions
id PK	id PK	id PK
store_id FK	store_id FK	store_id FK
category_id FK	name	status
barcode		start_date
stock_qty		end_date
purchase_items	purchases	suppliers
id PK	id PK	id PK
purchase_id FK	store_id FK	store_id FK
product_id FK	supplier_id FK	supplier_name
quantity	final_amount	phone
		balance
invoice_items	invoices	Customers
id PK	id PK	id PK
invoice_id FK	store_id FK	store_id FK
product_id FK	customer_id FK	name
quantity	final_amount	total_debt



شرح ERD

المتجر هو محور النظام. أغلب الجداول تحتوي store_id حتى تكون البيانات مفصولة بين المتاجر. الفواتير والديون والمدفوعات مرتبطة بالعملاء. المشتريات مرتبطة بالموردين. المنتجات مشتركة بين البيع والشراء.

14. جدول ملخص قاعدة البيانات

الجدول	الغرض	العلاقة الأهم
users	بيانات المستخدمين الأساسية	uid
stores	المتاجر المرتبطة بالمستخدمين	owner_id → users.uid
plans	خطط الاشتراك وحدود الاستخدام	مستقلة
subscriptions	اشتراك المتجر وحالته	store_id → stores.id
categories	تصنيفات المنتجات داخل المتجر	store_id → stores.id
products	المنتجات والباركود والمخزون	store_id, category_id
suppliers	الموردون وأرصدة المستحقات	store_id → stores.id
purchases	فواتير الشراء من الموردين	supplier_id, store_id
purchase_items	تفاصيل أصناف الشراء	purchase_id, product_id
Customers	العملاء وديونهم الإجمالية	store_id → stores.id
invoices	فواتير البيع	customer_id, store_id
invoice_items	تفاصيل أصناف الفاتورة	invoice_id, product_id
Debts	ديون العملاء الناتجة من الفواتير	invoice_id, customer_id
payment	دفعات العملاء	customer_id, store_id
debt_payments	توزيع الدفعات على الديون	payment_id, debt_id
sync_queue	طابور المزامنة المحلي	record_id + table_name

15. الخلاصة التنفيذية



هذا التصميم يعطي النظام قوة في السوق لأنه لا يتوقف عند انقطاع الإنترنت، ولا يضيع العمليات، ويمتلك أساسًا واضحًا للتوسع إلى فروع ومتاجر متعددة وخطط مدفوعة وصلاحيات مختلفة.